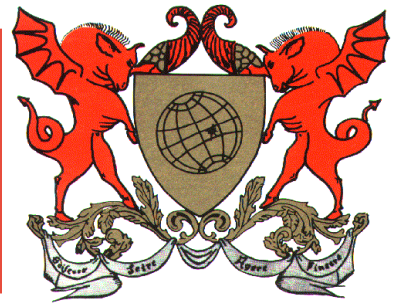
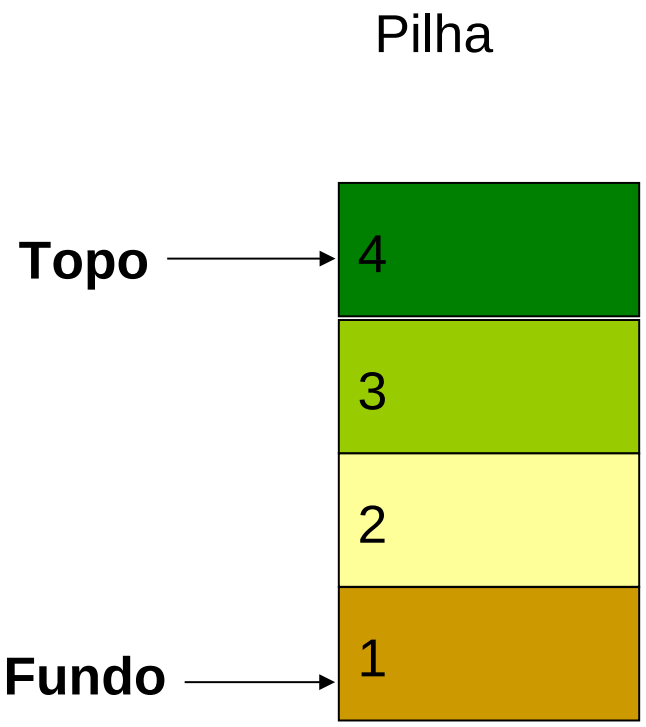


Universidade Federal de Viçosa
Campus de Florestal

Algoritmos e Estruturas de Dados I (CCF 211)

Pilhas (Cap03 – Seção 3.2- Ziviani)

O que é uma Pilha?



Pilha

- É uma **lista linear** em que todas as inserções, retiradas e, geralmente, todos os acessos são feitos em apenas um único e mesmo extremo
- Os itens são colocados uns sobre os outros. O item inserido mais recentemente está no topo e o inserido menos recentemente no fundo.
- O modelo intuitivo é o de um monte de pratos em uma prateleira, sendo conveniente retirar ou adicionar pratos na parte superior.

TAD Pilha

- Tipo Abstrato de dados Lista com a seguinte característica:

  O **último** elemento inserido é o **primeiro** a ser retirado/ removido

  (LIFO – *Last in First Out*)

- Analogia: pilha de pratos, livros, etc.

- Usos: Chamada de subprogramas, avalição de expressões aritméticas, etc.

TAD Pilha

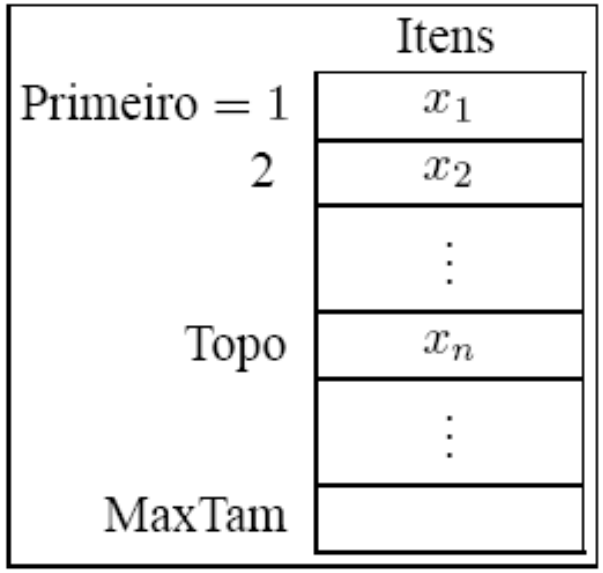
- Operações:
  - 1. FPVazia(Pilha). Faz a pilha ficar vazia.
  - 2. PEhVazia(Pilha). Retorna *true* se a pilha está vazia; caso contrário, retorna *false*.
  - 3. PEmpilha(Pilha, x). Insere o item x no topo da pilha.
  - 4. PDesempilha(Pilha, x). Retorna o item x no topo da pilha, retirando-o da pilha.
  - 5. PTamanho(Pilha). Esta função retorna o número de itens da pilha

- **As duas estruturas de dados mais utilizadas para representar pilhas são as implementações por meio de arranjos e lista encadeada**

Implementação de Pilhas através de Arranjos

- Os itens da pilha são armazenados em posições contíguas de memória.

- Como as inserções e as retiradas ocorrem no topo da pilha, um campo chamado Topo é utilizado para controlar a posição do item no topo da pilha (ou próxima posição disponível).



Estrutura da Pilha Usando Arranjo

```
#define MaxTam 1000

typedef int Apontador;
typedef int TChave;
typedef struct {
  TChave Chave;
  /* outros componentes */
} TItem;
typedef struct {
  TItem vItem[MaxTam];
  Apontador iTopo;
} TPilha;
```

Operações sobre Pilhas usando Arranjos

```
void FPVazia(TPilha* pPilha);

int PEhVazia(TPilha* pPilha);

int PEmpilha(TPilha* pPilha,TItem*  pItem);

int PDesempilha(TPilha* pPilha,TItem*  pItem);

int PTamanho(TipoPilha* pPilha);
```

Operações sobre Pilhas usando Arranjos

```
void FPVazia(TPilha* pPilha)
{
  pPilha->iTopo = 0;
} /* FPVazia */

int PEhVazia(TPilha* pPilha)
{
  return (pPilha->iTopo == 0);
} /* PEhVazia */
```

Operações sobre Pilhas usando Arranjos

```
int PEmpilha(TPilha* pPilha,
             TItem*  pItem)
{
  if (pPilha->iTopo == MaxTam)
    return 0;

  pPilha->vItem[pPilha->iTopo] = *pItem;
  pPilha->iTopo++;
  return 1;
} /* PEmpilha */
```

Operações sobre Pilhas usando Arranjos

```
int PDesempilha(TPilha* pPilha,
                TItem*  pItem)
{
  if (PEhVazia(pPilha))
    return 0;

  pPilha->iTopo--;
  *pItem = pPilha->vItem[pPilha->iTopo];
  return 1;
} /* PDesempilha */
```

Operações sobre Pilhas usando Arranjos

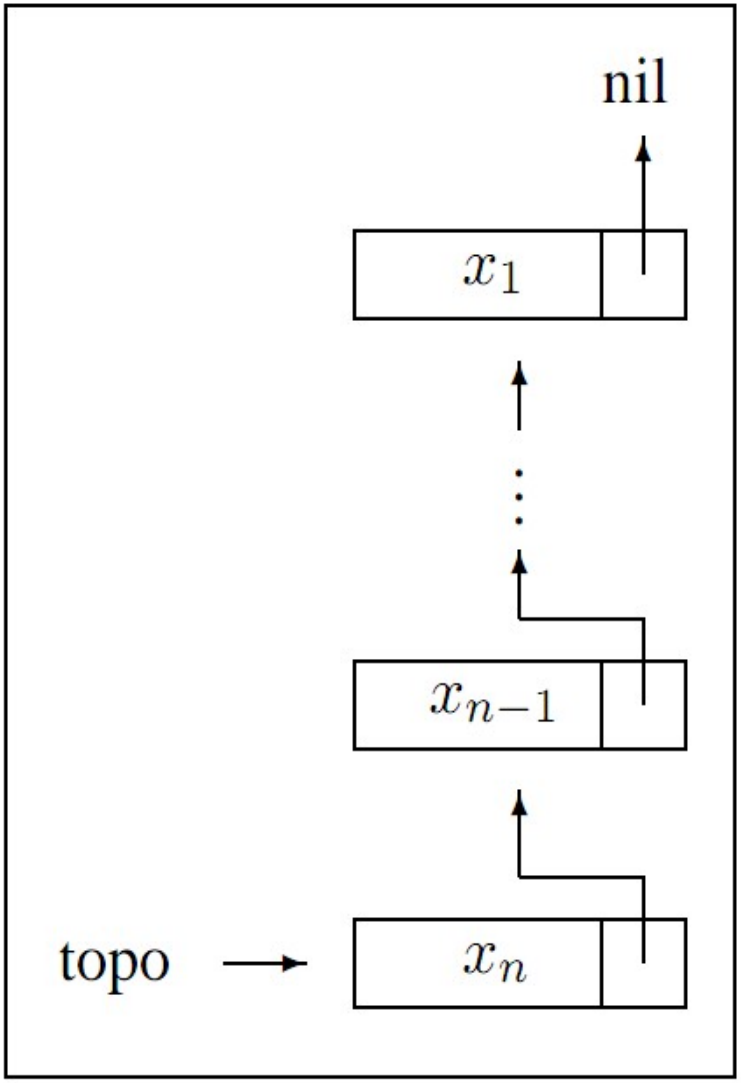
```
int PTamanho(TipoPilha* pPilha)
{
  return (pPilha->iTopo);
} /* Tamanho */
```

Implementação de Pilhas por meio de Lista Encadeada

- **Não há necessidade do uso de uma célula cabeça no topo para a implementação das operações empilha e desempilha.**

Para desempilhar o item x_n basta desligar a célula que contém x_n e a célula que contém x_{n-1} passa a ser a célula de topo.

- **Para empilhar um novo item, basta fazer a operação contrária, criando uma nova célula para receber o novo item.**



Estrutura da Pilha Usando Lista Encadeada

- **O campo Tamanho evita a contagem do número de itens na função Tamanho**
- **Cada célula de uma pilha contém um item da pilha e um apontador para outra célula**
- **O registro TPilha contém um apontador para o topo da pilha**

Estrutura da Pilha Usando Lista Encadeada

```
typedef int TChave;
typedef struct {
  TChave Chave;
  /* --- outros componentes --- */
} TItem;

typedef struct Celula* Apontador;
typedef struct Celula {
  TItem Item;
  struct Celula* pProx; // Apontador pProx
} TCelula;

typedef struct {
  Apontador pTopo;
  int iTamanho;
} TPilha;
```

Operações sobre Pilhas Usando Lista Encadeada

```
void FPVazia(TPilha* pPilha);

int PEhVazia(TPilha* pPilha);

int PEmpilha(TPilha* pPilha,TItem*  pItem);

int PDesempilha(TPilha* pPilha,TItem*  pItem);

int PTamanho(TipoPilha* pPilha);
```

Operações sobre Pilhas usando Lista Encadeada (sem cabeça)

```
void FPVazia(TPilha* pPilha)
{
  pPilha->pTopo  = NULL;
  pPilha->iTamanho = 0;
} /* FPVazia */

int PEhVazia(TPilha* pPilha)
{
  return (pPilha->pTopo == NULL);
} /* PEhVazia */
```

Operações sobre Pilhas usando Lista Encadeada (sem cabeça)

```
int PEmpilha(TPilha* pPilha,
             TItem*  pItem)
{
  Apontador pNovo;
  pNovo = (Apontador) malloc(sizeof(TCelula));
  if (pNovo == NULL)
    return 0;

  pNovo->Item = *pItem;
  pNovo->pProx = pPilha->pTopo;
  pPilha->pTopo = pNovo;
  pPilha->iTamanho++;
  return 1;
} /* PEmpilha */
```

Operações sobre Pilhas usando Lista Encadeada (sem cabeça)

```
int PDesempilha(TPilha* pPilha,
                TItem*  pItem)
{
  Apontador pAux; /* celula a ser removida */
  if (PEhVazia(pPilha))
    return 0;

  pAux = pPilha->pTopo;
  pPilha->pTopo = pAux->pProx;
  *pItem = pAux->Item;
  free(pAux);
  pPilha->iTamanho--;
  return 1;
} /* PDesempilha */
```

Operações sobre Pilhas usando Lista Encadeada (sem cabeça)

```
int PTamanho(TipoPilha* pPilha)
{
  return (pPilha->iTamanho);
} /* PTamanho */
```